

Python Automata Library (PAL) Cheat Sheet

Conventions

import PythonAutomataLibrary as pal · Safe mode is default. Call **pal.FastMode()** before constructing any PAL object. Put substantive model kernels under **@pal.njit(cache=True)**. **x...** means **x,y** in 2D and **x,y,z** in 3D. **Box(x1,x2,...)** lists lower/upper bounds for each axis; upper bounds are exclusive. Wrapped out-of-range coordinates wrap; nonwrapped ones are skipped.

Create state

pal.NewGrid(dims, dtype) — Create a typed lattice storing one value per site.
pal.NewAgentGrid(dims, numAgentProps=0, isStackable=False) — Create agents with optional properties; **dims=()** makes a nonspatial grid.
pal.NewPopGrid(dims, capacity=None) — Create integer counts per site; capacity limits total population.
pal.NewPDEgrid(dims) — Create a continuous field for PDE updates.
pal.NewList() — Create a mutable integer list for collecting/reusing query results.
pal.NewMultinomial() — Create a sampler for binomial/multinomial population draws.
Dimensions — 1–3 axes; a negative dimension wraps that axis, e.g. **(-nx, ny)**.

Shared lattice geometry

Properties — **xDim/yDim/zDim, nDims, wrapX/Y/Z**; a negative dimension enables wrapping on that axis.
ToI(x...) — Convert 1D/2D/3D coordinates into one linear site index.
ItoX/Y/Z(i) — Convert a linear site index back to its axis coordinate.
Regions — **Box(x1,x2,...)** selects a rectangular region; each axis has a lower bound and exclusive upper bound.
Neighborhoods — **pal.MooreHood(dim, excludeCenter=False)** · **pal.VonNeumannHood(dim, excludeCenter=False)** · **pal.CircleHood(dim, rad, excludeCenter=False)**. These return relative offsets, not indices; pass them to **grid.Hood(hood, x[,y,z])**. Set **excludeCenter=True** to omit the center.

Grid / common indexing

g[i] — Read or write a site by linear index.
g[x...] — Read or write a site using dimensional coordinates.
Slices — Return detached NumPy copies, not live views; editing a slice does not update the grid.
Pattern — **g = pal.NewGrid((40,40), float)** · **g[10,12] = 1.0** · **v = g[10,12]**; supported dtypes include bool, fixed-width integers, and float32/float64.

Draw / output

Pixels — **pix, win = pal.StartPixWindow(xDim,yDim,scale=1,title='PAL',headless=False)**; set **pix[x,y]=RGB**; **win.Update()**; **win.Save(path, block=True)**; **win.Close()**.
GIF — **StartGif(path,delay=100)** · **AddGifFrame(block=False)** · **StopGif()**; call **Update()** before capture.
OpenGL — **pal.StartOpenGLWindow(...)**; draw with **Circle, Box, BoxSQ, Line, Borders**; scene controls include **Camera, Background, Clear**.

Lists + randomness

List methods — **Append(i)** adds an integer; **Clear()** empties the list; **Random()** picks an entry; **Shuffle()** reorders entries; **All()** copies the list; **Iter()** traverses without copying.
RNG — **pal.Seed(seed)** · **pal.Random()** · **pal.RandInt(n)** → integer **0..n-1**; seed once for repeatable runs. PAL calls share a call-order-dependent random stream distinct from NumPy's RNG.
Multinomial — **m.Binomial(n,p)**; **Setup(...)** then **Sample(...)** for repeated multinomial draws.

AgentGrid

Create / move — **NewAgentSQ(x,y)** / **MoveSQ(a,x,y)** use lattice coordinates; **NewAgent(x,y)** / **Move(a,x,y)** use continuous positions. In 2D/3D, SQ forms take coordinates (or a linear index where supported); do not mix the two position systems.
Agent state — **grid[a,p]** reads/writes property **p**; **I(a)** is linear site index; **XSQ/YSQ/ZSQ(a)** are lattice coordinates; **X/Y/Z(a)** are continuous coordinates. Check **Alive(a)** before use when agents may be disposed; **Dispose(a)** removes an agent.
Queries — **GetPop()**, **AgentsAt(...)**, **LastAgent(...)**, **AgentsInRadius(...)**, **counts[...]**.
Iteration — for **a in agents.All()**: ... iterates a detached snapshot and is safe for structural mutation such as **Dispose**. **AgentsInRadius(...)** returns nearby agents; wrapped displacements account for periodic boundaries.
Wrapping — **DispWrapX/Y/Z(p1,p2)** gives wrapped displacement.

PopGrid + PDEgrid

Transactional update — **Add(v, ...)** accumulates pending changes; **Update()** applies them together. **Reset()** clears both current and pending state. Direct **grid[...] = v** changes current state immediately, outside the transaction.
Population — **pop.GetPop()** total; **pop.All()** nonzero site indices. **capacity** limits total population.
PDE setup — **field.SetTimeSpaceStep(dt, dx[,dy,dz])** sets time and spatial steps before updates. Use spacing values that satisfy the selected scheme's stability requirements.
Diffusion — **Diffusion, DiffusionMask, DiffusionField, DiffusionInterfaces, DiffusionADI**; radial 1D: **DiffusionRadialCircle/Sphere**.
Advection — **Advection, AdvectionField, AdvectionInterfaces**.
More detail: [Manual](#) · [API Guide](#) · [API Reference \(Documentation/\)](#).